



# UNIVERSIDAD TECNOLÓGICA DE QUERÉTARO

Ingeniería  
En Desarrollo y Gestión de Software

Seguridad en el desarrollo de aplicaciones

Grupo IDGS17

Nombre del alumno:  
Jovanny Guadalupe López Cebreros

Matricula:  
2023371119

Profesor:  
Gerardo Ramírez Villareal

Práctica 1.

## Objetivo

Investigar las 10 vulnerabilidades más críticas según OWASP Top 10 2023, comprender los mecanismos de ataque y su impacto en la seguridad de aplicaciones, y documentar casos reales de cada tipo de vulnerabilidad con ejemplos de código vulnerable y su versión segura.

### 1. Tabla de Vulnerabilidades OWASP Top 10 (2021/2023)

La siguiente tabla resume las 10 categorías más críticas de riesgos en aplicaciones web según la lista oficial de OWASP. Esta lista, aunque fue publicada formalmente en 2021, sigue siendo la referencia vigente en la industria y es ampliamente reconocida como el estándar de seguridad para desarrolladores web.

| ID  | Nombre                                        | Descripción                                                                                    | Ejemplo de Ataque                                                                     | Impacto Potencial                                                   | Mitigación                                                                              |
|-----|-----------------------------------------------|------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| A01 | Control de Acceso Roto(Broken Access Control) | Faltan restricciones sobre lo que los usuarios autenticados pueden hacer.                      | Un usuario normal cambia la URL perfil?id=1 a perfil?id=2 y ve datos de otro usuario. | Robo de datos de otros usuarios o toma de control de cuentas admin. | Validar permisos en el backend para cada petición, no confiar en la URL.                |
| A02 | Fallas Criptográficas                         | Datos sensibles como contraseñas o tarjetas se transmiten o guardan sin encriptar.             | Un atacante intercepta tráfico HTTP y roba contraseñas en texto plano.                | Exposición masiva de datos sensibles y robo de identidad.           | Usar HTTPS siempre y encriptar contraseñas con algoritmos fuertes como bcrypt.          |
| A03 | Inyección (Injection)                         | El sistema ejecuta código malicioso porque no limpió los datos enviados por el usuario.        | Poner ' OR 1=1 -- en un formulario de login para entrar sin contraseña.               | Pérdida total de la base de datos o control del servidor.           | Usar consultas parametrizadas (Prepared Statements) en lugar de concatenar texto.       |
| A04 | Diseño Inseguro                               | Fallas en la arquitectura desde la planeación, aunque el código esté bien implementado.        | Un sistema de recuperación de cuentas, hace preguntas muy fáciles de adivinar.        | Compromiso de cuentas enteras de manera sistemática.                | Modelado de amenazas desde el principio y mejores lógicas de negocio.                   |
| A05 | Configuración de Seguridad Incorrecta         | Dejar contraseñas por defecto, mensajes de error detallados o servicios innecesarios abiertos. | Acceder al panel de admin usando admin/admin porque nadie cambió las credenciales.    | Acceso no autorizado a servidores o bases de datos completas.       | Cambiar contraseñas por defecto y no mostrar errores de base de datos al usuario final. |
| A06 | Componentes Vulnerables y                     | Usar librerías, frameworks o sistemas                                                          | Usar una versión vieja de un plugin                                                   | El atacante toma el control del sitio usando                        | Mantener las dependencias                                                               |

| ID  | Nombre                                                    | Descripción                                                                      | Ejemplo de Ataque                                                                                             | Impacto Potencial                                                            | Mitigación                                                                             |
|-----|-----------------------------------------------------------|----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
|     | Desactualizados                                           | operativos que tienen vulnerabilidades conocidas.                                | de JS que permite hackear la web.                                                                             | exploits ya conocidos.                                                       | actualizadas y borrar las que no se usen.                                              |
| A07 | Fallas de Identificación y Autenticación                  | Errores en cómo se manejan las sesiones de los usuarios o las contraseñas.       | Ataques de fuerza bruta donde se prueban miles de contraseñas hasta acertar.                                  | Robo masivo de cuentas de usuario.                                           | Implementar MFA (Autenticación Multifactor) y bloquear cuentas tras intentos fallidos. |
| A08 | Fallas en la Integridad de Software y Datos               | El software se actualiza o despliega usando fuentes no confiables sin verificar. | Un hacker altera un parche de actualización y todos los clientes lo instalan.                                 | Infección masiva de la infraestructura y de los clientes.                    | Firmar digitalmente el código y verificar las descargas antes de instalarlas.          |
| A09 | Fallas en el Registro y Monitoreo                         | No guardar logs (registros) de quién hace qué, o no revisarlos.                  | Un hacker intenta entrar miles de veces durante meses y nadie se da cuenta.                                   | Daño prolongado sin que la empresa sepa que está siendo atacada.             | Guardar logs de todos los logins fallidos y usar alertas automáticas.                  |
| A10 | Falsificación de Solicitudes del Lado del Servidor (SSRF) | El servidor descarga información de una URL que el usuario envía, sin validarla. | El atacante obliga al servidor a leer archivos internos enviando la URL <code>http://localhost/admin</code> . | Acceso a redes internas a las que el atacante no tendría acceso normalmente. | Validar estrictamente las URLs enviadas por los usuarios mediante listas blancas.      |

## 2. Casos Reales por Vulnerabilidad

A continuación se documentan dos casos reales de cada categoría, basados en información pública disponible en reportes de seguridad, artículos especializados y comunicados oficiales. El objetivo es únicamente educativo.

| Vulnerabilidad                    | Empresa Afectada                    | Año  | Cómo fue explotada                                                                                                           |
|-----------------------------------|-------------------------------------|------|------------------------------------------------------------------------------------------------------------------------------|
| A01 – Control de Acceso Roto      | First American Financial            | 2019 | Cualquier persona podía ver documentos bancarios de otros usuarios solo cambiando el número en la URL.                       |
| A01 – Control de Acceso Roto      | Snapchat                            | 2013 | Un fallo en su API permitió saltar los controles de acceso y recopilar millones de números de teléfono y usuarios.           |
| A02 – Fallas Criptográficas       | Equifax                             | 2017 | Parte de los datos robados no estaban cifrados en las bases internas, facilitando el robo de millones de seguros sociales.   |
| A02 – Fallas Criptográficas       | Yahoo                               | 2013 | Contraseñas filtradas con MD5, un algoritmo de cifrado ya obsoleto y fácil de romper en ese entonces.                        |
| A03 – Inyección SQL               | Sony Pictures / PlayStation Network | 2011 | Inyección SQL clásica que expuso datos personales de millones de usuarios.                                                   |
| A03 – Inyección SQL               | TalkTalk                            | 2015 | Un adolescente usó SQL Injection básico para robar datos de clientes por falta de validación de entradas.                    |
| A04 – Diseño Inseguro             | Zoom                                | 2020 | Su diseño por defecto no requería contraseñas para reuniones, generando el fenómeno del 'Zoombombing'.                       |
| A04 – Diseño Inseguro             | Tesla                               | 2014 | La API de sus autos no tenía límite de intentos de login (rate limiting), permitiendo ataques de fuerza bruta.               |
| A05 – Config. Incorrecta          | Capital One                         | 2019 | Un WAF (firewall de aplicaciones web) mal configurado en AWS expuso datos de más de 100 millones de clientes.                |
| A05 – Config. Incorrecta          | Twitch                              | 2021 | Un error de configuración en un servidor expuso todo el código fuente y los pagos de los creadores de contenido.             |
| A06 – Componentes Desactualizados | Equifax                             | 2017 | El origen de la brecha fue no actualizar una vulnerabilidad conocida en el framework Apache Struts a tiempo.                 |
| A06 – Componentes Desactualizados | Panama Papers (Mossack Fonseca)     | 2016 | Se cree que fueron hackeados a través de un plugin desactualizado en su sitio WordPress o Drupal.                            |
| A07 – Fallas de Autenticación     | Nintendo                            | 2020 | Ataque de Credential Stuffing usando contraseñas robadas de otros sitios para entrar a cientos de miles de cuentas.          |
| A07 – Fallas de Autenticación     | Spotify                             | 2020 | Miles de cuentas comprometidas por hackers probando credenciales filtradas sin que hubiera protección contra automatización. |

| Vulnerabilidad             | Empresa Afectada                | Año  | Cómo fue explotada                                                                                                                     |
|----------------------------|---------------------------------|------|----------------------------------------------------------------------------------------------------------------------------------------|
| A08 – Fallas de Integridad | SolarWinds                      | 2020 | Hackers alteraron una actualización oficial del software Orion; miles de empresas la instalaron, dando acceso a redes gubernamentales. |
| A08 – Fallas de Integridad | ASUS (ShadowHammer)             | 2019 | Atacantes comprometieron la herramienta de actualización de ASUS para distribuir malware firmado digitalmente.                         |
| A09 – Fallas de Monitoreo  | Target                          | 2013 | Sus sistemas detectaron el malware y enviaron alertas, pero el equipo de seguridad las ignoró durante semanas.                         |
| A09 – Fallas de Monitoreo  | Marriott / Starwood             | 2018 | Los atacantes estuvieron robando datos silenciosamente durante aproximadamente 4 años sin ser detectados.                              |
| A10 – SSRF                 | Capital One                     | 2019 | Se usó SSRF para engañar al servidor y obtener las credenciales del servicio de metadatos de AWS.                                      |
| A10 – SSRF                 | Microsoft Exchange (ProxyLogon) | 2021 | El SSRF fue clave para que los atacantes falsificaran solicitudes como si fueran el servidor legítimo.                                 |

### 3. Código Vulnerable vs. Código Seguro

Para las tres vulnerabilidades que más impacto pueden tener en el desarrollo cotidiano de un estudiante de software, se presenta a continuación una comparación entre implementaciones inseguras y su versión corregida en JavaScript/pseudocódigo.

#### 3.1 A03 – Inyección SQL

Código vulnerable (concatenación directa de strings):

```
// ¡Peligro! Si el usuario escribe ' OR 1=1 -- puede saltarse la autenticación
let username = req.body.username;
let query = "SELECT * FROM users WHERE user = " + username + """;
db.execute(query);
```

Código seguro (consultas parametrizadas — Prepared Statements):

```
// El signo ? actúa como marcador; la BD trata la variable como texto, nunca como código
let username = req.body.username;
let query = "SELECT * FROM users WHERE user = ?";
db.execute(query, [username]);
```

#### 3.2 A01 – Control de Acceso Roto (IDOR)

Código vulnerable (sin validación de permisos):

```
// Recibe el ID directamente de la URL y no verifica si el usuario es el dueño
let id_cuenta = req.query.id;
let datos = db.get("SELECT * FROM cuentas WHERE id = ?", [id_cuenta]);
return datos; // Cualquier usuario puede ver la cuenta de cualquier otro
```

Código seguro (validación de propiedad en el backend):

```
let id_cuenta_solicitada = req.query.id;
let id_usuario_logueado = session.user.id; // Obtenido del token, no de la URL

if (id_cuenta_solicitada == id_usuario_logueado || session.user.role === 'admin') {
  let datos = db.get("SELECT * FROM cuentas WHERE id = ?", [id_cuenta_solicitada]);
  return datos;
} else {
  return "Acceso denegado"; // 403 Forbidden
}
```

#### 3.3 A07 – Fallas de Autenticación (contraseñas en texto plano)

Código vulnerable (guardar contraseña sin cifrar):

```
let password = req.body.password;
// Si la BD es comprometida, todas las contraseñas quedan expuestas directamente
db.execute("INSERT INTO users (user, password) VALUES (?, ?)", [user, password]);
```

Código seguro (usando hash con bcrypt):

```
const bcrypt = require('bcrypt');
let password = req.body.password;

// bcrypt genera un hash seguro con un 'factor de costo' de 10 rondas
let hashedPassword = bcrypt.hashSync(password, 10);
db.execute("INSERT INTO users (user, password) VALUES (?, ?)", [user, hashedPassword]);
// Aunque la BD sea robada, los hashes no revelan las contraseñas originales
```

## 4. Ranking Personal de Criticidad

A continuación presento las 5 vulnerabilidades que, desde mi perspectiva como estudiante de segundo cuatrimestre, considero más críticas. El criterio principal es qué tan probable es que yo mismo cometa ese error en mis proyectos actuales.

| P<br>os | Vulnerabilidad                    | Justificación                                                                                                                                                                                                                               |
|---------|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1°      | A03 – Inyección SQL               | Es el error más fácil de cometer cuando aprendemos a hacer consultas a bases de datos y solo concatenamos variables para que 'funcione rápido'. Un pequeño descuido puede comprometer toda la base de datos.                                |
| 2°      | A01 – Control de Acceso Roto      | A nivel de código la aplicación no marca ningún error, funciona perfectamente, pero por un descuido en la lógica de permisos permitimos que los usuarios vean información que no les corresponde. Es invisible si no lo buscas activamente. |
| 3°      | A06 – Componentes Desactualizados | En nuestros proyectos usamos npm install sin revisar exactamente qué librería y qué versión estamos bajando. Si instalamos dependencias viejas con vulnerabilidades conocidas, le hacemos el trabajo muy fácil a los atacantes.             |
| 4°      | A05 – Configuración Incorrecta    | Para pruebas locales solemos dejar puertos abiertos, contraseñas como 'root' o mensajes de error completos que después olvidamos corregir antes de subir a producción. Ese descuido puede costarle caro a un cliente.                       |
| 5°      | A07 – Fallas de Autenticación     | Guardar contraseñas en texto plano o no implementar protección contra fuerza bruta es un error grave porque expone directamente a las personas que confían en nuestro sistema con sus datos personales.                                     |

## 5. Conclusión

Antes de investigar esto para la práctica, mi mentalidad era básicamente: si el código corre y no truena, está bien. Ahora me parece un poco ingenuo haberlo pensado así, pero creo que es lo que le pasa a la mayoría en el primer año: el objetivo es que funcione, no que sea seguro.

Lo que más me impactó fue ver los casos reales. Equifax no fue hackeado con algo sofisticado; dejaron sin actualizar una librería que ya tenía parche publicado. Target tenía el sistema de alertas encendido, lo detectó, y alguien simplemente no hizo nada con esa alerta durante semanas. Eso me parece más preocupante que cualquier técnica de hacking: los errores organizacionales que pasan por descuido o por no darle prioridad al tema.

En mis propios proyectos del cuatrimestre he hecho varias de las cosas que aparecen en la lista de vulnerabilidades. Concatené strings en queries porque "era más rápido". Dejé errores de SQL visibles en el navegador para depurar más fácil. Nunca me pregunté si el usuario que hace una petición tiene permiso de verdad para hacer eso, o si solo validé del lado del cliente. No lo hice con mala intención, simplemente no lo tenía en el radar.

Eso es lo que me llevo de esta práctica: la seguridad no es un módulo separado que se estudia al final de la carrera. Es una pregunta que se hace mientras se escribe el código. ¿Este dato viene del usuario? Entonces no confíes en él. ¿Esta librería tiene cuánto tiempo sin actualización? ¿Quién puede acceder a esta ruta y lo estoy validando en el servidor, no solo en el botón?, No creo que a partir de mañana vaya a hacer todo perfecto, pero sí voy a hacer preguntas que antes no me hacía.